

Lab Experiment: Logistic Regression vs KNN on the Breast Cancer Dataset

Aim: train Logistic Regression and KNN on the Breast Cancer Wisconsin (Diagnostic) dataset and see which one classifies better.

Step 0 — Imports

```
In [1]:

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    confusion_matrix, ConfusionMatrixDisplay, classification_report
)

sns.set_style("whitegrid")
RANDOM_STATE = 42
```

Task 1 — Load the dataset

```
In [2]:

data = load_breast_cancer()

print("Dataset loaded successfully.")
print(f"Number of samples : {data.data.shape[0]}")
print(f"Number of features : {data.data.shape[1]}")
print(f"Target classes      : {list(data.target_names)}")

Dataset loaded successfully.
Number of samples : 569
Number of features : 30
Target classes      : [np.str_('malignant'), np.str_('benign')]
```

Task 2 — Convert to a DataFrame and take a first look

```
In [3]:

df = pd.DataFrame(data.data, columns=data.feature_names)
df["target"] = data.target
df["diagnosis"] = df["target"].map({0: "malignant", 1: "benign"})
df.head()
```

Out[3]:

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	mean concave points	mean symmetry	mean fractal dimension	...	w perim
0	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	0.14710	0.2419	0.07871	...	18
1	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	0.07017	0.1812	0.05667	...	15
2	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	0.12790	0.2069	0.05999	...	15
3	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	0.10520	0.2597	0.09744	...	9
4	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	0.10430	0.1809	0.05883	...	15

5 rows x 32 columns

In [4]:

df.describe().T

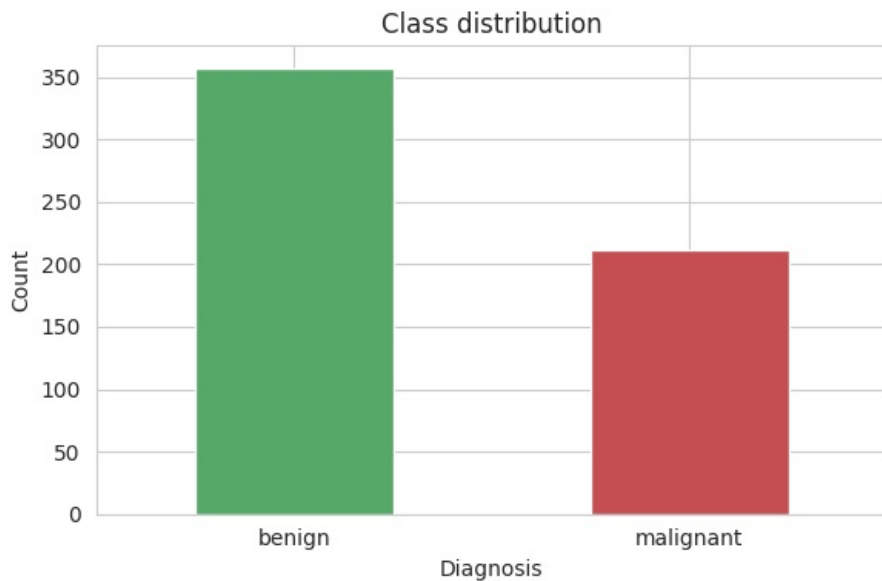
Out[4]:

	count	mean	std	min	25%	50%	75%	max
mean radius	569.0	14.127292	3.524049	6.981000	11.700000	13.370000	15.780000	28.11000
mean texture	569.0	19.289649	4.301036	9.710000	16.170000	18.840000	21.800000	39.28000
mean perimeter	569.0	91.969033	24.298981	43.790000	75.170000	86.240000	104.100000	188.50000
mean area	569.0	654.889104	351.914129	143.500000	420.300000	551.100000	782.700000	2501.00000
mean smoothness	569.0	0.096360	0.014064	0.052630	0.086370	0.095870	0.105300	0.16340
mean compactness	569.0	0.104341	0.052813	0.019380	0.064920	0.092630	0.130400	0.34540
mean concavity	569.0	0.088799	0.079720	0.000000	0.029560	0.061540	0.130700	0.42680
mean concave points	569.0	0.048919	0.038803	0.000000	0.020310	0.033500	0.074000	0.20120
mean symmetry	569.0	0.181162	0.027414	0.106000	0.161900	0.179200	0.195700	0.30400
mean fractal dimension	569.0	0.062798	0.007060	0.049960	0.057700	0.061540	0.066120	0.09744
radius error	569.0	0.405172	0.277313	0.111500	0.232400	0.324200	0.478900	2.87300
texture error	569.0	1.216853	0.551648	0.360200	0.833900	1.108000	1.474000	4.88500
perimeter error	569.0	2.866059	2.021855	0.757000	1.606000	2.287000	3.357000	21.98000
area error	569.0	40.337079	45.491006	6.802000	17.850000	24.530000	45.190000	542.20000
smoothness error	569.0	0.007041	0.003003	0.001713	0.005169	0.006380	0.008146	0.03113
compactness error	569.0	0.025478	0.017908	0.002252	0.013080	0.020450	0.032450	0.13540
concavity error	569.0	0.031894	0.030186	0.000000	0.015090	0.025890	0.042050	0.39600
concave points error	569.0	0.011796	0.006170	0.000000	0.007638	0.010930	0.014710	0.05279
symmetry error	569.0	0.020542	0.008266	0.007882	0.015160	0.018730	0.023480	0.07895
fractal dimension error	569.0	0.003795	0.002646	0.000895	0.002248	0.003187	0.004558	0.02984
worst radius	569.0	16.269190	4.833242	7.930000	13.010000	14.970000	18.790000	36.04000
worst texture	569.0	25.677223	6.146258	12.020000	21.080000	25.410000	29.720000	49.54000
worst perimeter	569.0	107.261213	33.602542	50.410000	84.110000	97.660000	125.400000	251.20000
worst area	569.0	880.583128	569.356993	185.200000	515.300000	686.500000	1084.000000	4254.00000
worst smoothness	569.0	0.132369	0.022832	0.071170	0.116600	0.131300	0.146000	0.22260
worst compactness	569.0	0.254265	0.157336	0.027290	0.147200	0.211900	0.339100	1.05800
worst concavity	569.0	0.272188	0.208624	0.000000	0.114500	0.226700	0.382900	1.25200
worst concave points	569.0	0.114606	0.065732	0.000000	0.064930	0.099930	0.161400	0.29100
worst symmetry	569.0	0.290076	0.061867	0.156500	0.250400	0.282200	0.317900	0.66380
worst fractal dimension	569.0	0.083946	0.018061	0.055040	0.071460	0.080040	0.092080	0.20750
target	569.0	0.627417	0.483918	0.000000	0.000000	1.000000	1.000000	1.00000

Observation: 30 continuous features, all derived from cell nuclei measurements, and on very different scales — mean area runs into the hundreds while mean smoothness sits around 0.1.

In [5]:

```
plt.figure(figsize=(6, 4))
df["diagnosis"].value_counts().plot(kind="bar", color=["#55A868", "#C44E52"])
plt.title("Class distribution")
plt.xlabel("Diagnosis")
plt.ylabel("Count")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```



Observation: the classes are moderately imbalanced — about 63% benign vs 37% malignant — worth keeping in mind since accuracy alone can be misleading here.

Task 3 — Check for missing values and duplicates

In [6]:

```
print("Missing values:", df.isnull().sum().sum())
print("Duplicate rows :", df.duplicated().sum())
```

Missing values: 0
Duplicate rows : 0

Observation: no missing values and no duplicates, so the dataset is ready to go without any cleaning.

Task 4 — Train/test split and feature scaling

In [7]:

```
X = df[data.feature_names].values
y = df["target"].values

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=RANDOM_STATE, stratify=y
)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print(f"Training samples: {X_train.shape[0]}")
print(f"Testing samples : {X_test.shape[0]}")
```

Training samples: 455
Testing samples : 114

Observation: using an 80:20 stratified split so the malignant:benign ratio stays consistent in both sets, and scaling only after the split to avoid leaking test data into the scaler.

Task 5 — Train the Logistic Regression classifier

In [8]:

```
log_reg = LogisticRegression(random_state=RANDOM_STATE, max_iter=1000)
log_reg.fit(X_train_scaled, y_train)

y_pred_lr = log_reg.predict(X_test_scaled)
y_proba_lr = log_reg.predict_proba(X_test_scaled)[:, 1]

print("Logistic Regression trained.")
```

Logistic Regression trained.

Task 6 — Train the KNN classifier

In [9]:

```
k_heuristic = int(round(np.sqrt(X_train.shape[0])))
if k_heuristic % 2 == 0:
    k_heuristic += 1 # keep it odd to avoid tie votes

knn = KNeighborsClassifier(n_neighbors=k_heuristic)
knn.fit(X_train_scaled, y_train)

y_pred_knn = knn.predict(X_test_scaled)
y_proba_knn = knn.predict_proba(X_test_scaled)[:, 1]

print(f"KNN trained with K = {k_heuristic} (chosen via the sqrt(n) heuristic).")
```

KNN trained with K = 21 (chosen via the sqrt(n) heuristic).

Observation: K was picked with the usual $\sqrt{n}$ rule of thumb rather than tuned, since the point of this lab is the model comparison, not squeezing out the last bit of KNN accuracy.

Task 7 — Evaluate both classifiers

In [10]:

```
def get_metrics(y_true, y_pred, model_name):
    return {
        "Model": model_name,
        "Accuracy": accuracy_score(y_true, y_pred),
        "Precision": precision_score(y_true, y_pred),
        "Recall": recall_score(y_true, y_pred),
        "F1 Score": f1_score(y_true, y_pred),
    }

lr_metrics = get_metrics(y_test, y_pred_lr, "Logistic Regression")
knn_metrics = get_metrics(y_test, y_pred_knn, "KNN")

metrics_df = pd.DataFrame([lr_metrics, knn_metrics]).set_index("Model").round(4)
metrics_df
```

Out[10]:

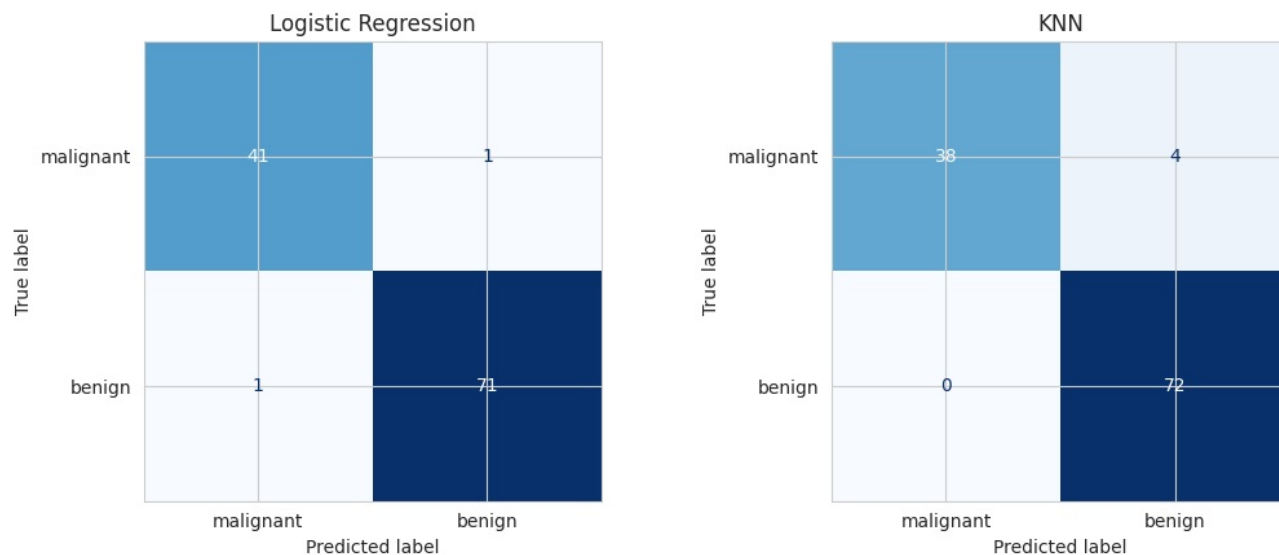
	Accuracy	Precision	Recall	F1 Score
Model				
Logistic Regression	0.9825	0.9861	0.9861	0.9861
KNN	0.9649	0.9474	1.0000	0.9730

In [11]:

```
fig, axes = plt.subplots(1, 2, figsize=(11, 4.5))

for ax, y_pred, title in zip(
    axes, [y_pred_lr, y_pred_knn], ["Logistic Regression", "KNN"]
):
    cm = confusion_matrix(y_test, y_pred)
    disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=data.target_names)
    disp.plot(ax=ax, colorbar=False, cmap="Blues")
    ax.set_title(title)

plt.tight_layout()
plt.show()
```



Observation: the confusion matrices show which errors each model actually makes — the false negatives (malignant predicted as benign) matter more clinically than false positives, since a missed cancer is far costlier than a follow-up test.

In [12]:

```
print("Logistic Regression\n" + "-"*45)
print(classification_report(y_test, y_pred_lr, target_names=data.target_names))

print("\nKNN\n" + "-"*45)
print(classification_report(y_test, y_pred_knn, target_names=data.target_names))
```

Logistic Regression

```
-----
              precision    recall  f1-score   support

 malignant      0.98      0.98      0.98        42
   benign      0.99      0.99      0.99        72

 accuracy              0.98        114
 macro avg      0.98      0.98      0.98        114
weighted avg      0.98      0.98      0.98        114
```

KNN

```
-----
              precision    recall  f1-score   support

 malignant      1.00      0.90      0.95        42
   benign      0.95      1.00      0.97        72

 accuracy              0.96        114
 macro avg      0.97      0.95      0.96        114
weighted avg      0.97      0.96      0.96        114
```

Task 8 — Comparison table

In [13]:

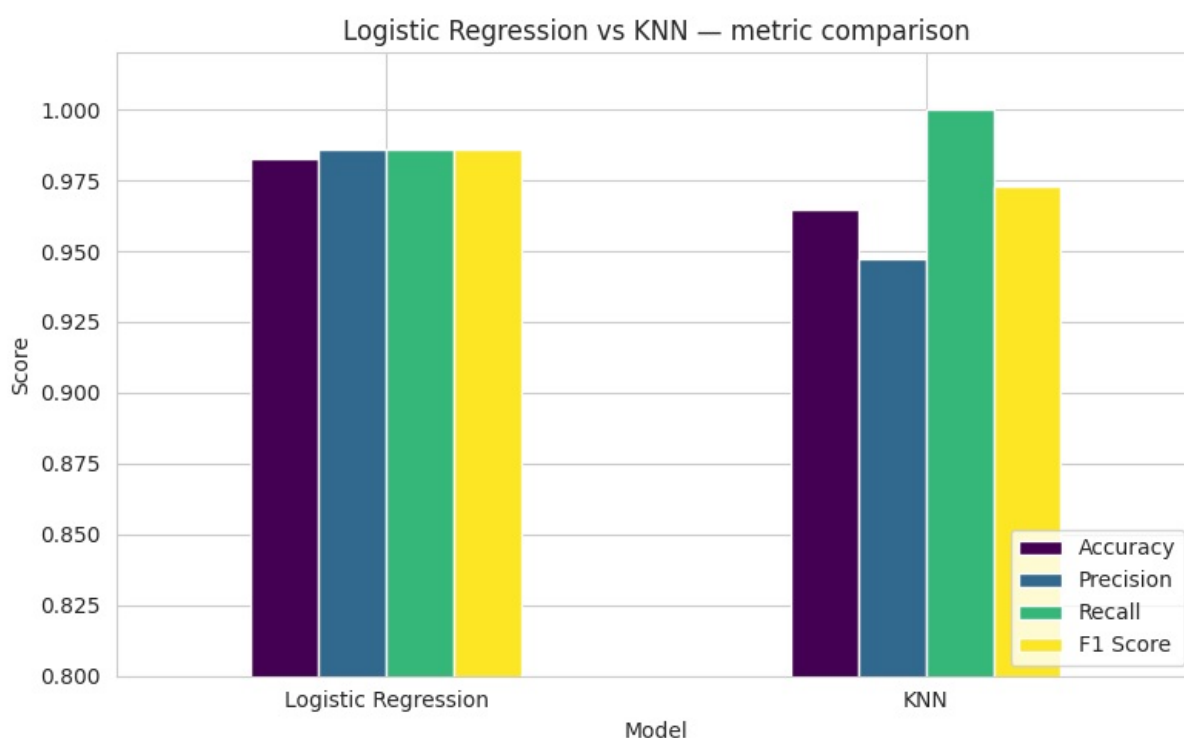
```
comparison_df = metrics_df.copy()
comparison_df["Best"] = comparison_df.idxmax()
comparison_df
```

Out[13]:

	Accuracy	Precision	Recall	F1 Score	Best
Model					
Logistic Regression	0.9825	0.9861	0.9861	0.9861	NaN
KNN	0.9649	0.9474	1.0000	0.9730	NaN

In [14]:

```
metrics_df.plot(kind="bar", figsize=(8, 5), colormap="viridis", rot=0)
plt.title("Logistic Regression vs KNN – metric comparison")
plt.ylabel("Score")
plt.ylim(0.8, 1.02)
plt.legend(loc="lower right")
plt.tight_layout()
plt.show()
```



Observation: the two models land close to each other on every metric, with Logistic Regression edging ahead slightly — not surprising, since these features are fairly linearly separable after scaling.

Conclusion

Better classifier: Logistic Regression performed marginally better on this dataset, likely because the scaled features already separate the classes in a roughly linear way, which plays to its strengths rather than KNN's.

KNN's trade-off: KNN is more sensitive to the choice of K and the local density of points, so its performance here is solid but has more room to move depending on tuning, whereas Logistic Regression gave a strong result with almost no tuning at all.